الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يونيورسيتي إسلام انتارابغسا ملايسيا

(Company No.: 198301005860)

Garden of Knowledge and Virtue

MECHATRONICS SYSTEM INTEGRATION : MCTA 3203

LABORATORY REPORT

SECTION 1

TITLE : PLC interfacing with microcontroller and PC over Ethernet/IP (WEEK 7)

LECTURER'S NAME: ASSOC. PROF. IR. TS. DR. ZULKIFLI BIN ZAINAL ABIDIN

DATE OF EXPERIMENT : 19 NOVEMBER 2025

DATE OF SUBMISSION : 3 DECEMBER 2025

GROUP : 4

NO.	NAME	MATRIC NO.
1	MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	2311779
2	AHMAD HARITH IMRAN BIN MOHD YUSOF	2311581
3	ARIFA AQILAH BINTI ABDUL HALIM	2311530

ACKNOWLEDGEMENTS

We would like to express our heartfelt appreciation to Dr. Zulkifli Bin Zainal Abidin for his continuous guidance, support, and insightful explanations throughout this experiment. His expertise greatly helped us understand the concepts of interfacing and digital control using Arduino. We would also like to thank our lab assistant for providing technical assistance and ensuring that the experiment ran smoothly. Finally, we extend our gratitude to our classmates for their cooperation and teamwork during the lab session.

ABSTRACT

The purpose of this experiment was to connect industrial control concepts with microcontroller-based systems by integrating a PLC and an Arduino through the OpenPLC Editor. The OpenPLC Editor is a free, open-source platform used to design ladder diagrams that follow the IEC 61131-3 standard.

The procedure involved developing two main ladder programs. The first was a simple blinking LED sequence, which was later improved using TON and TOF timer blocks for more accurate timing. The second was a Start-Stop Control Circuit created using latching logic. A critical step was Physical Addressing, which required mapping variables to the Arduino's physical pins using specific addresses. After testing the logic through simulation, the compiled code was uploaded to the Arduino using the appropriate COM port.

The results showed that ladder logic which is commonly used in industrial automation can be effectively implemented on a low-cost microcontroller platform. This highlights the importance of accurate hardware addressing and thorough simulation when moving from software design to real-time physical system control.

TABLE OF CONTENT

1.0 INTRODUCTION

This lab module focuses on integrating a PLC, a microcontroller, and a PC over Ethernet/IP with the main aim of using the OpenPLC Editor to program an Arduino for basic industrial control tasks. The OpenPLC Editor is used to create and simulate ladder logic before transferring it to the Arduino. In this session, we work toward understanding the OpenPLC interface, creating a simple blinking LED ladder program, mapping program variables to the Arduino's physical I/O pins, and implementing a Start-Stop Control Circuit using latching logic.

A PLC is an industrial computer designed for automating electromechanical processes and the OpenPLC Editor supports languages such as Ladder Diagram, where logic is represented through power rails, contacts, and coils. Core concepts applied in this module include the use of TON and TOF timers to control the LED's blinking interval, latching logic to maintain an output after the start button is released and physical addressing using notations like %IX0.0 and %QX0.0 to connect program variables to actual Arduino pins.

The hypothesis is that ladder logic created in the OpenPLC Editor can be compiled and executed reliably on an Arduino running the OpenPLC Runtime. It is expected that the LED blinking program will operate as designed, that timer modifications will allow precise adjustment of the blinking interval and that the Start-Stop Control Circuit will perform correct latching behavior, keeping the LED ON after the start button is pressed and turning it OFF only when the stop button is activated.

2.0 MATERIAL AND EQUIPMENT

1. OpenPLC Editor software
2. Arduino Board
3. 2 Push Button Switches
4. LED
5. Resistors
6. Jumper Wires
7. Breadboard

3.0 EXPERIMENTAL SETUP

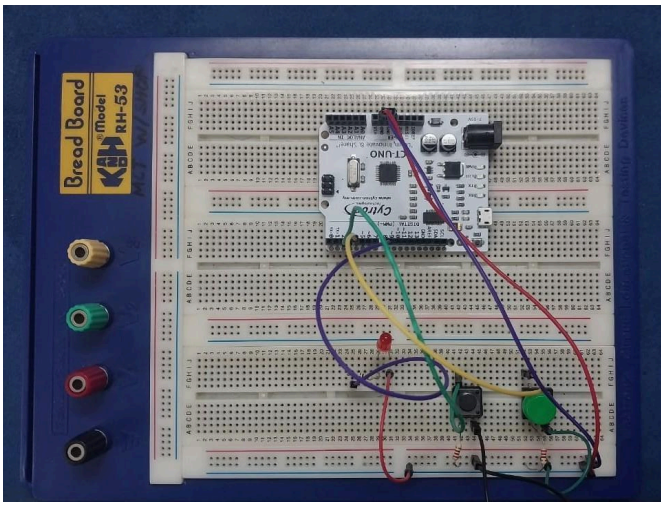


Figure A: Components connected to arduino

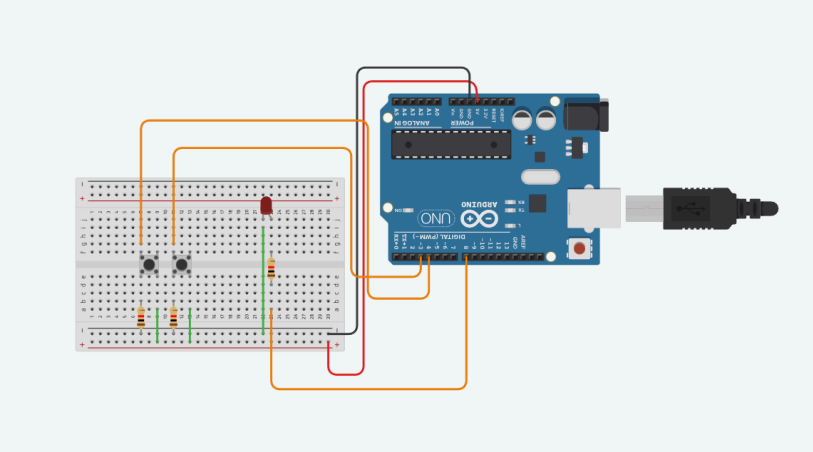


Figure B: Components connected to arduino board in Tinkercad

1. The pin is connected as below;
    - Button 1: Pin 3, GND and VCC to GND and 5v of Arduino
    - Button 2: Pin 4, GND and VCC to GND and 5v of Arduino
    - LED : LED pin connected to pin 8 of Arduino
2. Open up the PLC editor

3. Simulate the task

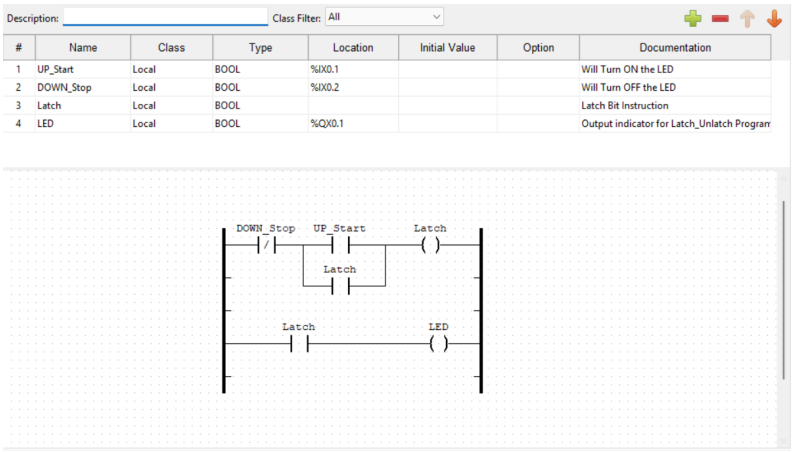


Figure C: Components connection using OpenPLC editor

4. Transfer the program to PLC which is the arduino uno

4.0 METHODOLOGY

Ladder Diagram Development

- The OpenPLC Editor was launched and a new project was created.
- Required Boolean variables were declared according to their functions:
    - **Blinking LED task**: led
    - **Start–Stop control**: UP_Start, DOWN_Stop, Latch, LED
- Ladder diagrams were built for all tasks:
    - **Blinking LED:** Basic rung using a negated contact and coil.
    - **Timer Exercise:** Added TON and TOF blocks with preset T#500ms to extend blinking duration.

    - **Start–Stop Control:** Constructed with normally open Start, normally closed Stop, seal-in (latching) contact, and output coil.
- Together, these diagrams formed the logical framework for LED blinking, timer control, and Start–Stop operation.

Simulation and Verification

- Each ladder diagram was compiled to check for syntax or logic errors.
- The OpenPLC simulator was used to observe behavior:
    - Blinking LED toggled correctly.
    - Timer delays produced the expected extended blinking cycle.
    - Start–Stop logic: pressing Start latched the LED ON, pressing Stop turned it OFF.

Hardware Configuration and Program Upload

- The Arduino Uno was set as the PLC hardware.
- Variables were mapped to physical addresses using OpenPLC's pin-mapping format:
    - UP_Start → %IX0.1 (Pin 3)
    - DOWN_Stop → %IX0.2 (Pin 4)
    - LED → %QX0.1 (Pin 8)
- The correct COM port was identified.

- The compiled ladder program was transferred to the Arduino using the Transfer to PLC tool.
- Successful transfer confirmed the microcontroller was executing the intended logic

Circuit Assembly and Testing

- The circuits were assembled on a breadboard using push buttons, resistors, jumpers, and an LED connected to the assigned Arduino pins.
- Wiring followed the physical address assignments: Start and Stop buttons to input pins, LED to output pin.
- Functionality tests were performed:
    - LED blinked according to the programmed timer sequence.
    - Pressing Start activated and latched the LED.
    - Pressing Stop turned the LED off.
- Any wiring, addressing, or connection issues were corrected during testing to ensure full operational reliability.

5.0 DATA COLLECTION

This lab experiment involves components like Arduino Uno as the PLC, two buttons and an LED light. This combination of components is used to demonstrate how the PLC works without the complication of coding in the IDE. Below is the table showing the variables used:

#	Name	Class	Type	Location	Documentation
1	UP_Start (Button1)	Local	BOOL	%IX0.1 (PIN 3)	Will Turn ON the LED
2	DOWN_Stop (Button2)	Local	BOOL	%IX0.2 (PIN 4)	Will Turn OFF the LED
3	Latch	Local	BOOL		Latch Bit Instruction
4	LED	Local	BOOL	%QX0.1 (PIN 8)	Output indicator for Latch_Unlatch Program

6.0 DATA ANALYSIS

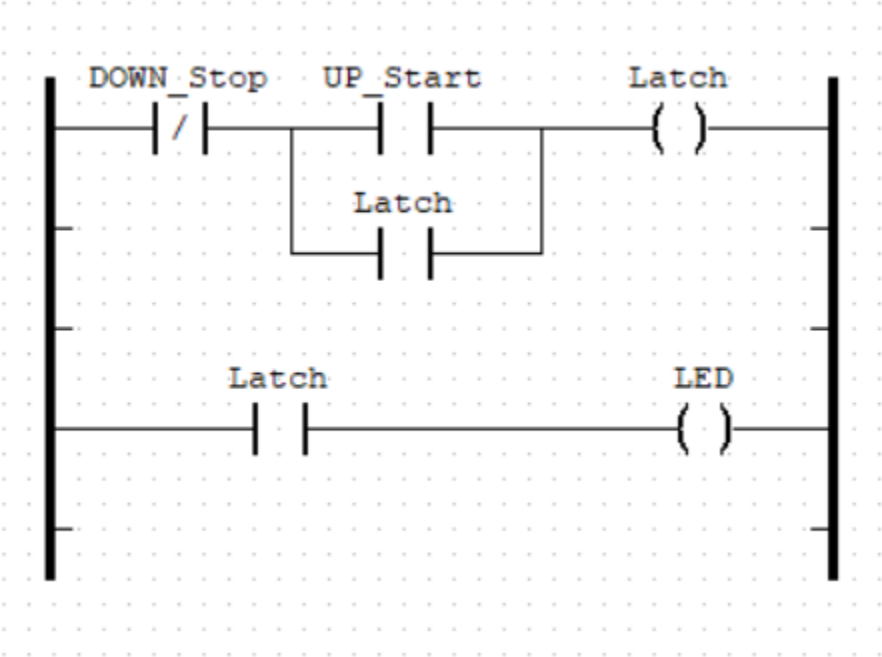


Figure D: Ladder diagram for the experiment

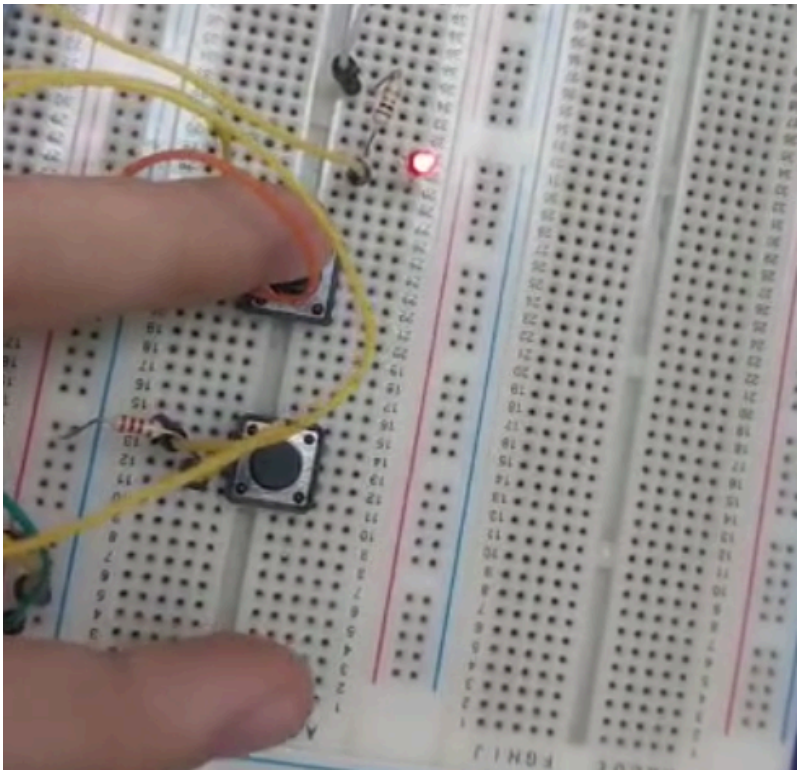
The first rung

- When button 2 is not pressed,the line will run towards button 1 which is Normally Open.
- When button 1 is pressed, the line will activate Latch and set it to true.
- To make sure, the line still runs even though button 1 is false, latch below the Up_Start will activate as long as button 2 is not pressed.

The second rung

- Sequentially, when the latch is rung on the first line, this will activate the latch on the second rung and pass current towards the LED to be true and turned ON.

7.0 RESULTS



The Start–Stop control circuit was successfully developed, simulated, and implemented using the OpenPLC Editor and Arduino board. The ladder diagram created in OpenPLC compiled without errors, and the simulation confirmed the expected behavior: pressing the START button energized the output coil, causing the LED to turn on and remain latched even after the button was released. Pressing the STOP button immediately de-energized the output coil, turning the LED off.

After transferring the compiled program to the Arduino, the hardware test produced results consistent with the simulation. When the circuit was wired according to the pin configuration, the system responded correctly:

- The START push button activated the LED and latched the output.

- The STOP push button reliably broke the latch and turned the LED off.

- No unintended LED behavior (flickering, random switching) was observed, indicating correct variable assignment and proper pin configuration.

Overall, the experiment demonstrated that the OpenPLC ladder logic—once uploaded to the Arduino—successfully replicated a standard industrial Start-Stop control circuit operation. The findings validate that PLC-style control can be implemented effectively using low-cost microcontroller hardware.

8.0 DISCUSSION

The experiment effectively demonstrated the integration of industrial control logic (Ladder Diagram) onto a cost-effective and versatile microcontroller platform (Arduino) using the OpenPLC Editor and OpenPLC Runtime. The core objective—designing, simulating, and executing control programs—was successfully achieved.

The successful implementation of the blinking LED circuit confirmed that the OpenPLC Runtime can reliably manage simple, repetitive outputs. Furthermore, the inclusion of TON (Timer ON-delay) and TOF (Timer OFF-delay) blocks illustrated the system's ability to handle precise time-dependent events, a key requirement in automated control systems.

Most importantly, the Start-Stop Control Circuit verified that the system could implement latching logic, maintaining output states such as LEDs or motors even after a momentary start input is released. The use of physical addressing (e.g; mapping UP_Start to %IX0.1 and LED to %QX0.0) effectively linked abstract ladder logic symbols to the specific Arduino I/O pins. These results highlight that the OpenPLC platform provides a robust, free, and open-source solution for learning and prototyping IEC 61131-3 compliant PLC applications without the need for expensive dedicated PLC hardware.

Sources of Error and Limitations

1. **Switch Bounce:** Mechanical push buttons (PB1, PB2) naturally exhibit contact bounce, which cause rapid on/off toggling. While industrial PLCs filter this, it could affect behavior in this setup.
2. **Incorrect Physical Addressing:** Misassigning a variable's Location (e.g; %IX0.0) could prevent inputs or outputs from functioning correctly.
3. **Wiring Errors:** Misplacement of components on the breadboard such as improper connections for pull-down resistors (R1, R2) or the current-limiting resistor (R3) could result in floating inputs or LED failure.

9.0 CONCLUSION

The experiment demonstrated that ladder logic designed using the OpenPLC Editor can be successfully simulated and executed on an Arduino microcontroller, confirming the system's ability to perform fundamental industrial control tasks such as LED blinking, timer-based operations, and Start–Stop latching control. The results showed consistent and accurate behavior across both software simulation and physical hardware testing, highlighting the importance of correct variable mapping, proper pin addressing, and careful wiring.

The findings fully supported the original hypothesis, which proposed that ladder logic developed in the OpenPLC environment could operate reliably on an Arduino running the OpenPLC Runtime. Both the blinking LED program and the Start–Stop circuit performed exactly as intended, with timers functioning properly and the latching mechanism maintaining its state until interrupted.

These outcomes demonstrate the broader applicability of using OpenPLC with microcontrollers for educational, prototyping, and low-cost automation purposes. By replicating PLC functionality on accessible hardware, this approach provides a practical platform for learning industrial control principles, testing automation logic before deployment, and exploring real-world applications such as machine control, safety interlocks, and process automation.

10.0 RECOMMENDATIONS

For future iterations of this experiment, several improvements can be made to enhance accuracy, reliability, and overall learning value. One recommended improvement is the use of hardware debouncing circuits or software-based debouncing methods to minimize the effects of switch bounce on the Start–Stop inputs. Incorporating pull-down or pull-up resistors more systematically can also help stabilize input signals and prevent floating values. Additionally, using clearer labeling on the breadboard and systematically organizing jumper wires would reduce wiring errors that may cause unexpected behavior.

Future students may also benefit from spending additional time understanding physical addressing in OpenPLC, as incorrect mapping is one of the most common sources of errors. It is recommended that students verify each address and pin assignment before uploading the program. Simulating the ladder diagram multiple times before transferring it to the hardware is also helpful, as it allows errors to be identified early. Finally, exploring additional OpenPLC function blocks—such as counters, edge detectors, or analog inputs—would provide deeper insight into industrial control applications and broaden the practical skills gained from the experiment.

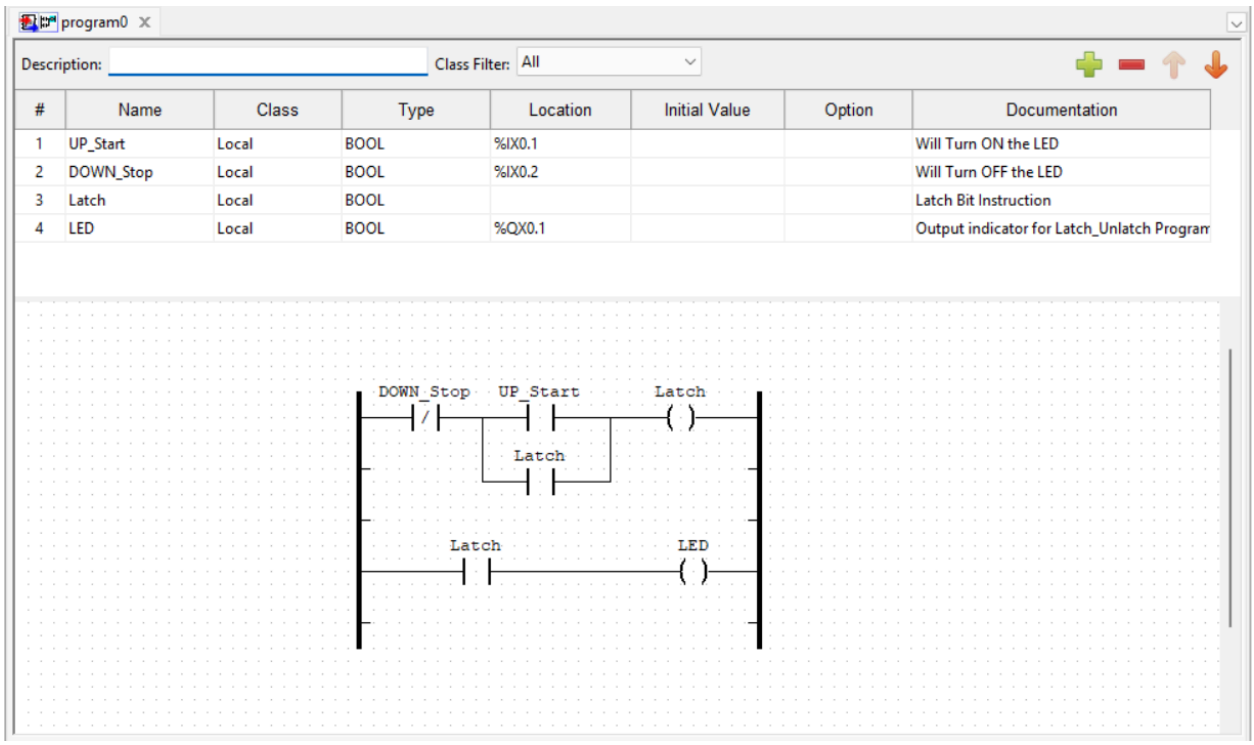
11.0 REFERENCES

Autonomy Logic. (2022, October 5). *Physical addressing*. Retrieved from https://autonomylogic.com/docs/2-4-physical-addressing/

Autonomy Logic. (2022, October 5). *Creating your first project on OpenPLC editor*. Retrieved from https://autonomylogic.com/docs/3-2-creating-your-first-project-on-openplc-editor/

12.0 APPENDICES

Appendix A (Ladder Diagram using OPEN PLC)



13.0 STUDENT'S DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

Signature: 	Read	/

Name: AHMAD HARITH IMRAN BIN MOHD YUSOF	Understand	/
Matric No.: 2311581	Agree	/
Contribution : Assembler, table of content, results, conclusion, recommendation and student declaration.		

Signature: 	Read	/
Name: MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	Understand	/
Matric No.: 2311779	Agree	/
Contribution : Coder, experimental setup, data collection, data analysis, references and appendices.		

Signature:	Read	/
Name: ARIFA AQILAH BINTI ABDUL HALIM	Understand	/
Matric No.: 2311530	Agree	/
Contribution : Assembler, abstract, introduction,material and equipment, methodology and discussion.		